CODE CRITIQUE

Expat NL Mortgage RAG vs. Rabobank GenAI Engineer Standards

A detailed technical assessment | Findings & Fix Roadmap

Prepared: March 2026 | Ajai

What the Role Demands

Key technical requirements extracted from the job description

Production-Grade Agents

Build GenAI agents from prototype to production rollout. Not demos — real systems.

Python Services

Build and maintain Python-based services integrating GenAI into internal platforms.

API & Event Integration

Connect agents to APIs, event systems, and developer tooling across the org.

Prompt Iteration & Eval

Improve quality through software + prompt iteration, evaluation, and user feedback.

Rapid Prototyping

Prototype working solutions often within hours or days to validate ideas.

Measurement & Value

Implement tools to measure the value delivered by GenAI tooling.

Overall Assessment

2.0

out of 5.0

Early prototype with strong architectural planning but immature implementation

Production Agent Design	2/5
Python Service Quality	2/5
Error Handling	1/5
Testing & Measurement	1/5
RAG Implementation	3/5
API Integration	3/5
Code Reusability	3/5
Documentation Quality	4/5
Prompt Eval & Iteration	1/5
Production Readiness	1/5

What Works Well

Architectural Planning

Excellent 4-phase roadmap in PHASES.md (690 lines). Clear progression from MVP to multi-agent system with well-defined milestones.

Hybrid RAG Search

Reciprocal Rank Fusion (RRF) combining vector + keyword search. Configurable rrf_k parameter. A solid foundation for retrieval quality.

Provider Abstraction

Flexible LLM provider layer supporting OpenAI, OpenRouter, and Ollama. Environment-driven config with no hardcoded API keys.

Documentation

Comprehensive DEPLOYMENT.md, PHASES.md, and .env.example. Responsible AI policy documented. Clear onboarding path for new developers.

CRITICAL

Silent Failures Everywhere

Bare except blocks swallow errors silently — the system fails without anyone knowing.

CURRENT — retrieval.py:44, 87

```
try:
    results = qdrant.search(...)
except Exception:
    return [], []   # Silent! No logging!
```

FIX — Log, report, degrade gracefully

```
try:
    results = qdrant.search(...)
except Exception as e:
    logger.error(f"Retrieval: {e}")
    return [], [{"error": str(e)}]
```

Same pattern found in 6+ locations:

`documents.py:102`	Deletes old chunks, no rollback on failure
`documents.py:111`	No retry on API rate limits (429)
`chunking.py:132`	LLM chunking silently degrades to simple mode
`app.py:74`	Tavily web search fails silently
`app.py:107`	No timeout on streaming calls (600s!)

CRITICAL

Testing & Evaluation Gap

57
lines of tests

1500+
lines of library code

~4%
test coverage (est.)

What's Missing

No integration tests

Qdrant, provider, and app flow never tested end-to-end

No PDF extraction tests

Core ingestion pipeline has zero test coverage

No edge case tests

Empty queries, very long text, rate limits, special characters untested

CI doesn't enforce quality

Linting uses '|| true' (ignores failures); no service containers for Qdrant

RAGAS evals are stubs

Golden dataset exists (data/golden_rag.json) but evaluation pipeline not wired

No type checking

No mypy configuration; heavy use of 'Any' type annotations

HIGH

Placeholder Code in Production Paths

Hardcoded formulas and rule-based logic that will mislead users or break at scale.

Mortgage Calculator (app.py:164-168)

Hardcoded: 0.45% monthly interest, 6% buyer costs.
Users making real financial decisions may trust these numbers.

FIX: Implement real ING formula OR add prominent disclaimer and disable by default.

Knowledge Graph (graph_kg.py:20-68)

Rule-based regex extraction — brittle and not extensible.
Hard-coded entity list requires code changes for new domains.

FIX: Replace with spaCy NER (nl_core_news_sm) or LLM-based entity extraction.

Agent Routing (agents.py:18-32)

Keyword matching for intent routing:
"Is the school near the house?" won't route to location.

FIX: Use LLM-based classifier or semantic similarity with intent exemplars.

UI Directives (a2ui.py:21-37)

Free-text matching: "I'll show you the calculator" triggers the show_calculator directive incorrectly.

FIX: Use structured JSON format from agent output, not free-form text matching.

Observability & Monitoring Gaps

Phase 3 goals are documented but implementation is incomplete across the board.

WHAT EXISTS

- Naive drift detection (monitoring/drift_detection.py)
- JSON-based metric persistence (portable)
- Placeholder RAGAS golden dataset
- Reference to Langfuse in DEPLOYMENT.md

WHAT'S MISSING

- Langfuse callback not wired in code
- No Prometheus /metrics endpoint active
- No Grafana dashboard configured
- Drift detection lacks statistical rigor
- No structured logging (JSON format)
- No circuit breakers for external APIs

Priority Fixes

- Wire Langfuse callback to LLM client calls for trace visibility
- Replace naive mean comparison with proper statistical tests (t-test, CUSUM)
- Add atomic file writes (write-then-rename) to prevent JSON corruption
- Implement structured JSON logging with python-json-logger

Code Quality & Technical Debt

7+
legacy app files

0
docstrings in lib/

Any
dominant type hint

Legacy File Cruft

app_phase1.py, app_wRAG.py, app_UploadPDF_Chat.py, and 4+ more variants clutter the repo. Only app.py is canonical.

FIX Delete all legacy variants. Keep app.py as single entry point.

No Docstrings

All lib/ modules lack Google-style docstrings. Hard to understand intent and expected behavior of functions.

FIX Add docstrings to all public functions with Args, Returns, and Raises sections.

Weak Type Hints

Heavy use of 'Any' throughout. No mypy config. IDE autocompletion fails; type errors caught at runtime only.

FIX Add explicit types. Configure mypy in CI. Replace 'Any' with proper QdrantClient, list[float], etc.

No Config Validation

.env.example lacks validation schema. Missing env vars discovered only at runtime with cryptic errors.

FIX Use Pydantic BaseSettings for typed config. Validate all required vars at startup.

Production Readiness Checklist

Measured against the job requirement: "from prototype to production rollout"

FAIL **Structured logging**

No logging.config; bare print/logger.warning

FAIL **Rate-limit handling (429, 503)**

No retry logic for API calls

FAIL **Graceful degradation**

Errors silently return empty results

FAIL **Session isolation**

No concurrent user handling

FAIL **Atomic write operations**

JSON metrics file can corrupt on concurrent access

FAIL **Health check endpoint**

No /health or readiness probe

FAIL **Request timeouts with backoff**

600s timeout on Ollama; no exponential backoff

FAIL **Circuit breakers for APIs**

If Qdrant/Tavily goes down, the whole app hangs

FAIL **Input validation**

No max length check, no sanitization

FAIL **Security scanning**

No bandit or safety checks in CI

FAIL **Max message history**

st.session_state['messages'] grows unbounded

PASS **Dependency pinning**

requirements.txt pins versions correctly

Fix Roadmap: Quick Wins

High-impact changes requiring minimal effort — do these first. (All achievable in under 1 week)

1. **Delete Legacy Files** — 30 min
   Remove 7+ app variants (app_phase1.py, app_wRAG.py, etc.). Keep app.py only.

2. **Add Error Logging** — 2 hours
   Replace all bare except blocks with logger.error() calls across 6 locations.

3. **Add Request Validation** — 1 hour
   Max input length (5000 chars), sanitize special characters in app.py.

4. **Add Request Timeouts** — 1 hour
   Change 600s timeout to 30s default. Add max_tokens to LLM calls.

5. **Fix CI Pipeline** — 2 hours
   Remove '|| true' from ruff check. Add Qdrant service container.

6. **Add Calculator Disclaimer** — 30 min
   Prominent warning that mortgage numbers are placeholder estimates only.

Fix Roadmap: Major Overhaul (2-4 Weeks)

Week 1-2: Robustness

- Implement exponential backoff (tenacity library) for all API calls
- Add circuit breakers for Qdrant, Tavily, Nominatim
- Replace keyword scoring with Qdrant sparse search (BM25)
- Wire Langfuse callback to LLM client calls
- Add 200+ lines of integration tests with Qdrant service

Week 2-3: Quality

- Add Pydantic BaseSettings for typed config validation
- Implement structured JSON logging (python-json-logger)
- Add mypy to CI pipeline; eliminate 'Any' types
- Add Google-style docstrings to all lib/ modules
- Implement RAGAS evals pipeline with golden dataset

Week 3-4: Intelligence

- Replace keyword agent routing with LLM-based intent detection
- Replace regex KG extraction with spaCy NER
- Add Prometheus metrics + Grafana dashboard
- Implement proper statistical drift detection
- Add message history limit + session isolation

Key Patterns to Adopt

Concrete code patterns that would bring this codebase closer to production-grade.

Retry with Backoff

```
from tenacity import retry,
  stop_after_attempt,
  wait_exponential

@retry(
  stop=stop_after_attempt(3),
  wait=wait_exponential(min=2)
)
def embed_batch(client, texts):
    ...
```

documents.py, location.py,
retrieval.py

Config Validation

```
from pydantic import BaseSettings

class Settings(BaseSettings):
  QDRANT_URL: str
  LLM_PROVIDER: str = "openai"
  OPENAI_API_KEY: str | None

  class Config:
    env_file = ".env"

settings = Settings()
```

All modules —
replace os.environ.get()

Structured Logging

```
import logging
from pythonjsonlogger import
  jsonlogger

handler = logging.StreamHandler()
handler.setFormatter(
  jsonlogger.JsonFormatter()
)
logger = logging.getLogger(__name__)
logger.addHandler(handler)
```

All modules —
replace bare print()

Summary & Next Steps

Strong Foundation

- Clean lib/ architecture
- Hybrid RRF search
- Flexible provider layer
- Excellent documentation
- Clear 4-phase roadmap

Critical Gaps

- Silent error handling
- Near-zero test coverage
- No observability wired
- Placeholder formulas
- No production hardening

Immediate Actions

- Delete 7+ legacy files
- Add logging everywhere
- Fix CI pipeline
- Add integration tests
- Wire Langfuse tracing

The architectural vision is sound — the PHASES.md roadmap shows real engineering thinking.
The gap is in execution: production hardening, testing rigor, and observability.
With 2-4 weeks of focused effort, this codebase can meet the Rabobank GenAI Engineer standard.